Computação em Nuvem

Sistema de entrega de comida baseado em cloud

Gustavo Tironi, Kauan Mariani Ferreira, Matheus Fillype Ferreira de Carvalho, Sillas Rocha da Costa

20 de abril de 2026

Conteúdo

1 Introdução **2**

2 Modelagem Geral **2**

3 Decisões de Projeto **6**

4 Testes de Carga **7**

5 Modelo de Custo **10**

6 Conclusão **11**

1 Introdução

Neste trabalho, propomos e implementamos um sistema de *delivery* de alimentos baseado em nuvem. Nosso objetivo é criar um serviço que comporte múltiplos tipos de usuários (clientes, restaurantes e entregadores) e orquestre, de maneira harmônica por meio da AWS, a interação entre eles. Todo o serviço é construído sobre APIs com recursos gerenciados na nuvem.

Buscamos modelar um sistema robusto a falhas isoladas de serviços, escalável e isolado em relação ao aumento de carga, aplicando os conceitos estudados na disciplina de Computação em Nuvem.

O sistema possui suporte local via Docker para desenvolvimento e suporte em nuvem pela AWS para produção. Além disso, desenvolvemos um *dashboard* proprietário para análise de carga nos serviços, testes de carga unitários para identificar gargalos em *endpoints* isolados e um simulador de pedidos para validar o sistema de ponta a ponta.

Ao longo deste relatório, explicamos e justificamos as decisões por trás da modelagem adotada, mostramos alguns testes de carga e garantias praticas dos requisitos não funcionais e, por fim, aprestamos um modelo de custo para esse projeto.

2 Modelagem Geral

A arquitetura do sistema foi estruturada em dois blocos lógicos principais: o Cluster Principal, responsável por toda a lógica de negócios e persistência de dados do sistema de delivery; e o Cluster Simuladores, encarregado de orquestrar a servir como porta de entrada das interações do usuário com o sistema. É por meio dele que interagimos para fazer os testes de carga, por exemplo. A Figura 1 ilustra a visão de alto nível da integração entre esses dois ambientes.

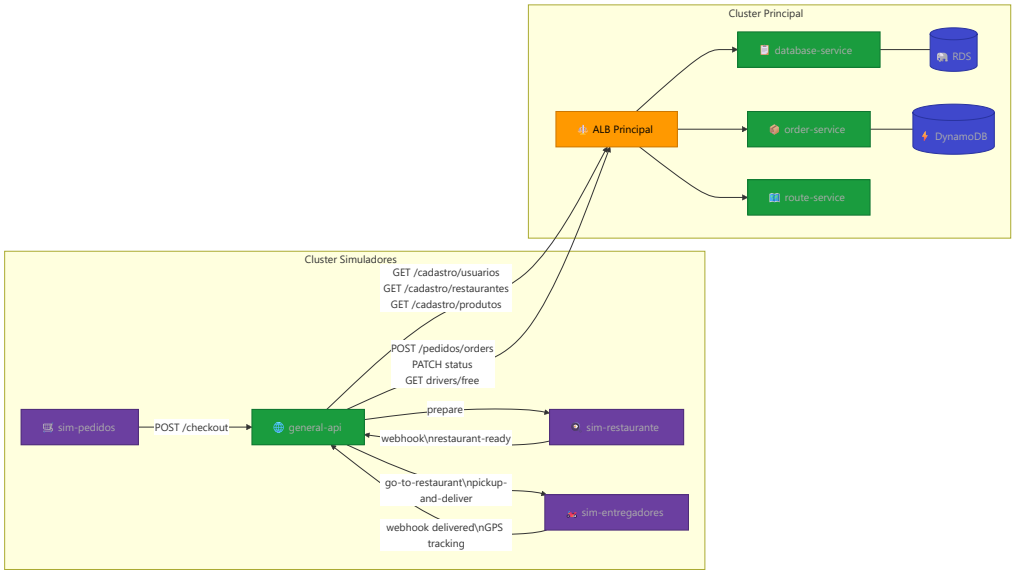

Figura 1: Visão geral da comunicação entre o Cluster de Simuladores e o Cluster Principal.

Essa separação permite que o sistema de testes opere de forma. A integração entre o simulador e o sistema ocorre de maneira padronizada via requisições API, na qual os simuladores imitam o comportamento de múltiplos usuários reais ao consumir as APIs expostas pelo balanceador de carga do cluster principal.

A seguir, detalhamos a composição e as responsabilidades específicas de cada um desses blocos.

Cluster Principal

Como ilustrado na Figura 2, o ponto de entrada para os usuários reais (Clientes) é um Application Load Balancer (ALB Principal). O ALB atua como um API Gateway reverso, realizando o roteamento do tráfego com base no path da requisição (URI) para três serviços independentes:

- **Base de dados:** Serviço responsável pelo gerenciamento de entidades estáticas (usuários, restaurantes, produtos e entregadores). Comunica-se de forma com um banco de dados relacional **Amazon RDS**. O acesso ao RDS é estritamente protegido por um Security Group interno.

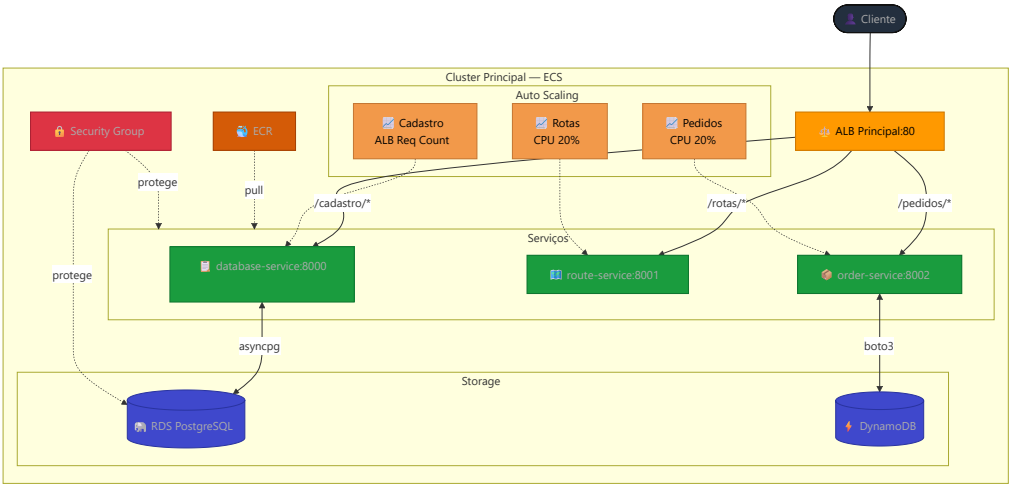

Figura 2: Arquitetura do Cluster Principal contendo o fluxo de requisições, serviços e recursos de armazenamento na AWS.

- **Route-service:** Serviço de processamento intensivo focado no cálculo de rotas (A*) e disponibilidade de entregadores.
- **Pedidos:** Serviço responsável por gerenciar o ciclo de vida e o checkout das entregas. Ele interage diretamente com o **Amazon DynamoDB**, garantindo baixa latência para transações primárias de leitura e escrita.

Todos os serviços operam de forma independente e buscam suas imagens de contêiner no Amazon Elastic Container Registry (ECR). Para lidar com a variação de demanda, as tasks do ECS estão sob políticas de Auto Scaling, escalando o serviço de rotas e pedidos com base no consumo de CPU (alvo de 20%) e o serviço de banco de dados com base na contagem de requisições do ALB.

Cluster de Simuladores

Para validar a resiliência do sistema, construímos um ambiente isolado (Cluster Simuladores), detalhado na Figura 3. A interface de controle é um Dashboard em Streamlit, que se comunica através de um ALB Interno com a infraestrutura de simulação.

Neste cluster, a general-api atua como o grande ponto de orquestração que interage com a

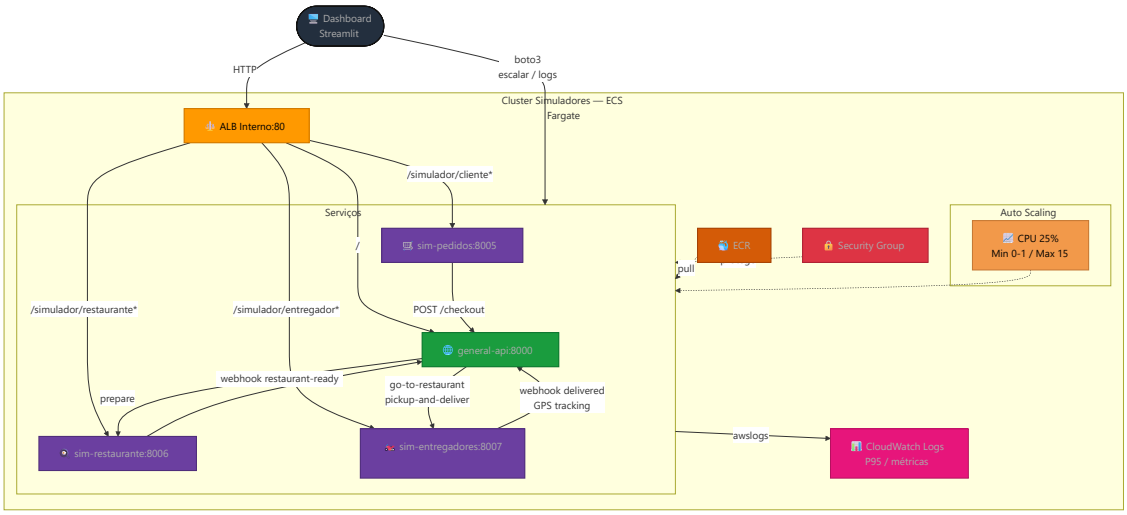

Figura 3: Arquitetura do Cluster de Simuladores e o fluxo interno de orquestração de testes de carga.

simulação dos usuários. Os demais componentes de teste (`sim-pedidos`, `sim-restaurante` e `sim-entregadores`) operam de forma estritamente interna e não interagem com o sistema principal de forma isolada; eles são utilizados internamente pelo sistema e comandados diretamente pela general-api.

Integração e Ciclo de Vida do Pedido

O fluxo de dados end-to-end orquestrado no simulador ocorre da seguinte maneira:

1. O processo é iniciado via `sim-pedidos`, que dispara o *checkout* para a `general-api`.
2. A `general-api` repassa a carga para o sistema real, consumindo os *endpoints* do ALB Principal (GET em cadastros, POST de criação de pedidos e GET para buscar motoristas livres no *route-service*).
3. Com o pedido criado, a `general-api` emite um comando interno (`prepare`) para o `sim-restaurante`, que, simulando o tempo de cozinha, devolve internamente um *webhook* (`restaurant-ready`).

4. Por fim, a `general-api` aciona o componente interno `sim-entregadores` (`pickup-and-deliver`). Este serviço emite atualizações a cada 100ms de localização e, ao concluir o trajeto, dispara o *webhook* `delivered`.

5. Todo esse ciclo interno da `general-api` se converte em ações no Cluster Principal, gerando, por exemplo, chamadas para atualizar o DynamoDB.

3 Decisões de Projeto

ECS Fargate

Utilizamos o ECS com Fargate como orquestrador de contêineres. Com ele, a AWS gerencia automaticamente o escalonamento da infraestrutura subjacente, sem necessidade de provisionar ou manter instâncias EC2. Escolhemos o Fargate pelo *setup* rápido e pelo modelo de cobrança por uso, eliminando o custo de máquinas ociosas por má configuração. Isso nos permite focar exclusivamente na otimização de vCPU e RAM por *task*. Para nosso caso de uso, com tráfego variável e escopo reduzido, o esforço adicional de gerenciar EC2 não se justifica.

Em todos os serviços ECS, utilizamos um *Application Load Balancer* (ALB) para distribuição de carga entre contêineres. O ALB é o mecanismo que viabiliza o escalonamento automático de forma simples, via *Target Tracking*. Para o serviço de rotas e gerenciamento de pedidos, utilizamos utilização de CPU das *tasks* como métrica de escala. Para o serviço de cadastro, utilizamos requisições por *target* no *Target Group*, por ser uma métrica mais representativa de sua carga.

Dimensionamento dos serviços ECS

Para o serviço de cadastro, utilizamos 256 de CPU e 512 MB de memória. Trata-se de um CRUD simples com baixa demanda computacional. Para o cálculo de rotas, utilizamos 1024 de CPU e 2048 MB, pois o processo é *compute-intensive*: além de executar o algoritmo A* no grafo viário, é necessário carregar o grafo completo da cidade em memória. Para o serviço de pedidos, apesar do alto volume de operações, as chamadas ao DynamoDB não são custosas e não consomem CPU de forma significativa, portanto, mantivemos 256 de CPU e 512 MB, igual ao cadastro.

RDS PostgreSQL

O RDS foi utilizado para dados de cadastro (usuários, restaurantes, entregadores e produtos), que exigem persistência relacional e são acessados de forma esporádica. Dados históricos (pedidos finalizados) também são enviados para persistência nesse banco após a conclusão da entrega. Isso resulta em uma carga previsível e baixa no RDS, enquanto as operações de alta frequência, como os pedidos ativos, localização e atualização de status, são delegadas ao DynamoDB.

Optamos por não utilizar Multi-AZ nem *Read Replica* nesta fase. A carga de leitura é baixa e alta disponibilidade não é um requisito crítico para o projeto atual. Em produção real, o Multi-AZ faria sentido: ao falhar a instância primária, a réplica assume automaticamente no mesmo *endpoint* em aproximadamente 60 segundos. O Amazon Aurora foi descartado por ser excessivo para nossa escala controlada.

Para o tipo de instância, escolhemos `db.t4g.small`. Uma instância *micro* poderia ser suficiente, mas optamos pela *small* para garantir margem de RAM, especialmente considerando o *overhead* de conexões do PostgreSQL. A família T4g foi preferida à T3 por oferecer melhor desempenho por custo. Uma otimização futura é validar empiricamente se o *micro* suporta a carga de teste.

DynamoDB

Todas as transações primárias do sistema ocorrem no DynamoDB: criação de pedidos, atualizações de status e rastreamento de localização dos entregadores, tanto em escrita quanto em leitura. O DynamoDB resolve exatamente esse problema, múltiplas requisições simultâneas de leitura e escrita com baixa latência e escalonamento automático. Aumentar a instância do RDS para absorver essa carga não faria sentido nem seria eficiente.

Além da latência reduzida, o DynamoDB confere robustez à parte mais crítica do negócio: criar e gerenciar pedidos. Mesmo que outros serviços não consigam escalar rapidamente o suficiente, o DynamoDB permanece estável, mantendo a experiência do cliente íntegra e abrindo tempo para que a arquitetura interna se ajuste à demanda.

4 Testes de Carga

Os testes de tolerância a falha de uma instância, pico isolado do restante do sistema e escalabilidade automática do sistema foram demonstrados na apresentação parcial. Portanto, focaremos nesta seção no teste de carga que faltava no nosso projeto. Avaliamos a

arquitetura sob três cenários distintos de requisições: 10, 50 e 200 pedidos por segundo. Cabe ressaltar que os arquivos de configuração utilizados para os testes da entrega parcial foram mantidos no repositório do projeto.

Iniciamos as medições com o cenário de 10 requisições por segundo (10/s). O sistema suportou bem. A latência P95 estabilizou em 164ms, mantendo-se confortavelmente abaixo da meta estipulada de $\leq$ 500ms.

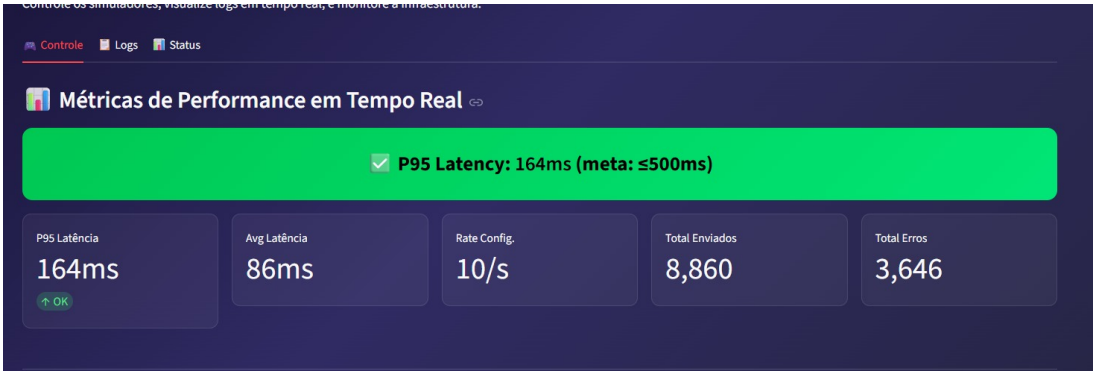

Figura 4: Métricas de performance para o cenário de carga de 10 pedidos por segundo, atingindo latência P95 de 164ms.

No teste de 50 pedidos por segundo, observamos um aumento na latência P95 para 281ms. Esse incremento era esperado, dado que elevamos a carga do sistema em cinco vezes. É importante mencionar que a infraestrutura demandou um tempo de aquecimento (*warm-up*) para estabilizar, algo que falaremos mais no detalhe mais adiante. A captura das métricas refletidas na Figura 5 foi realizada após esse período de estabilização, e não no momento zero da simulação. Nesse cenário, o escalonamento automático cumpriu seu papel e conseguiu manter a performance da aplicação dentro da meta.

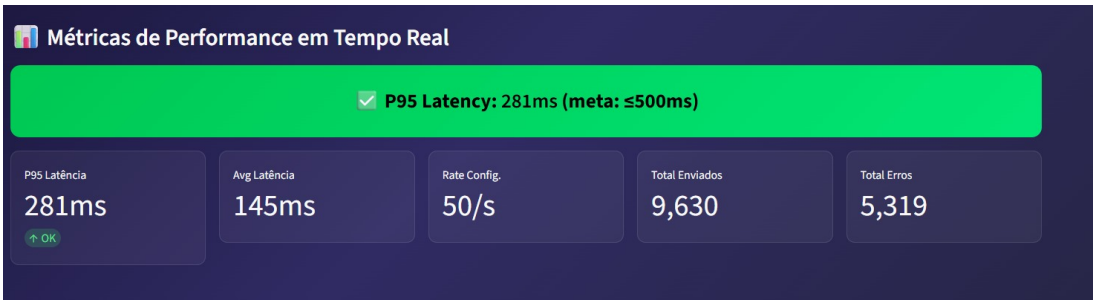

Figura 5: Métricas de performance para o cenário de carga de 50 pedidos por segundo, atingindo latência P95 de 281ms.

Avançando para o último e mais intenso teste, submetemos o sistema a 200 pedidos por segundo. Neste cenário de estresse, a infraestrutura foi levada ao seu limite e não foi possível manter a latência P95 abaixo da meta de 500ms, alcançando a marca de 2240ms.

Embora não seja o cenário ideal, consideramos o resultado tolerável para um volume de requisições tão extremo. Novamente, a métrica foi coletada após o tempo necessário para que o sistema se adequar à carga.

Figura 6: Métricas de performance para o cenário de estresse de 200 pedidos por segundo, resultando em latência P95 de 2240ms.

Análise dos resultados e problemas encontrados

Como pode ser notado nos *dashboards* das três imagens, há um volume considerável de transações contabilizadas como erro. Ao investigar a causa raiz, identificamos um problema de concorrência (*race condition*) na atribuição de entregadores. Basicamente, quando dois usuários realizam pedidos simultâneos para o mesmo restaurante (frequente em nossa simulação devido ao número limitado de restaurantes pré-cadastrados), o sistema tenta alocar o mesmo motorista para ambas as rotas. Isso resulta em falha no fechamento de um dos pedidos. Trata-se de uma melhoria arquitetural que o sistema precisará no futuro (como a implementação de bloqueios transacionais ou filas), mas que não invalidou a medição correta da latência de ponta a ponta das requisições que tiveram sucesso.

Outro comportamento observado é o atraso inerente do sistema para se adaptar a picos abruptos de carga. Ao iniciar os testes, a latência sobe consideravelmente até que, após alguns minutos, novas instâncias sejam provisionadas pela AWS e estabilizem o processamento em valores aceitáveis. Esse gargalo de inicialização poderia ser mitigado ajustando as métricas de forma mais agressiva. Contudo, consideramos o comportamento atual adequado para esta fase do projeto, ficando como proposta de otimização para uma versão futura (Talvez para a A2).

Por fim, esses resultados, somados aos testes exibidos na apresentação parcial, demonstram que conseguimos construir um sistema que, embora ainda não seja perfeito, está no caminho certo e chega muito próximo dos requisitos solicitados. Temos ciência de algumas melhorias possíveis e de certos defeitos que precisarão ser resolvidos em uma próxima versão. Contudo, entregamos hoje uma arquitetura minimamente funcional, apesar dos

bugs, algo comum ao se criar um software.

5 Modelo de Custo

Uma forma robusta de modelar o custo é usando a *AWS Pricing Calculator*[1], configurando cada serviço manualmente. Para uma abordagem mais programática, usamos também o *endpoint* de *pricing* do *boto3*, que permite buscar preços diretamente via API.

No nosso caso, vamos falar sobre cada serviço e então fazer uma estimativa. O **ECS Fargate** cobra por vCPU-hora e por GB de memória-hora consumidos pelas tasks em execução, ou seja, quanto mais tasks rodando, maior o custo. O **RDS** cobra por hora de instância independente da carga, mais o armazenamento em GB. Um detalhe importante: instâncias T4g e T3 operam em modo *unlimited*, ou seja, se a utilização média de CPU exceder a linha de base da instância em uma janela de 24 horas, os créditos excedentes são cobrados a $0,075 por vCPU-hora.[2] O **DynamoDB** no modelo sob demanda cobra por operação de leitura e escrita realizada — $0,625 por milhão de escritas e $0,125 por milhão de leituras. O **ALB** cobra por hora de existência mais por LCUs consumidas, que crescem com o volume de tráfego.

Para estimar os custos, definimos dois cenários: **operação normal**, com 20 pedidos/segundo por 8 horas diárias durante 30 dias (17M pedidos/mês); e **evento especial**, com 200 pedidos/segundo por 4 horas mais 200 pedidos/segundo por 8 horas em um único dia (30M pedidos/mês). Para o ECS, estimamos quantas tasks o auto scaling manteria em cada cenário, com mínimo de 2 e máximo de 10. Para o DynamoDB, estimamos 10 operações por pedido (criações, atualizações de status, rastreamento), sendo 60% escritas e 40% leituras.

Com isso, chegamos a um custo estimado de **$250/mês no cenário normal** e **$360/mês no evento especial**, um aumento de aproximadamente 55%. O principal driver do aumento é o DynamoDB, seguido do ALB, cujo custo de LCUs cresce proporcionalmente ao tráfego. O ECS e RDS não mudariam tanto.

Vale ressaltar que esses valores são uma **estimativa de ordem de grandeza**, não um orçamento preciso. O objetivo é comparar os cenários e identificar, no geral, quanto a nossa arquitetura gastaria.

[1] https://calculator.aws
[2] https://aws.amazon.com/pt/rds/pricing/

6 Conclusão

Este trabalho demonstrou a construção e implementação de um sistema de *delivery* de alimentos baseado em nuvem. Conseguimos criar uma infraestrutura escalável na AWS que aborda desafios reais, como o gerenciamento dinâmico de tráfego, a persistência de dados de alta concorrência e a orquestração eficiente entre múltiplos usuários (clientes, restaurantes e entregadores).

Mais do que isso, o desenvolvimento deste projeto foi essencial para **aprimorar nosso entendimento técnico sobre computação em nuvem**. A experiência com o Fargate mostrou o poder da conteinerização *serverless* para escalar serviços de forma independente e automática. O uso combinado de bancos de dados relacionais (RDS) para dados estáticos e NoSQL (DynamoDB) para operações transacionais rápidas reforçou a importância de escolher a ferramenta certa, dependendo da necessidade.

Com foco na AWS, percebemos a **complexidade de desenhar uma arquitetura** e de calibrar corretamente as regras de escalabilidade e escolher os tipos de instancia mais eficientes, minimizando o custo. Os testes de carga revelaram que a infraestrutura, embora robusta, possui um tempo de resposta inerente (o *warm-up* das instâncias) e que falhas de concorrência (*race conditions*) podem surgir rapidamente em cenários de alta demanda simultânea, algo que precisaria ser tratado. Contudo, o **simples fato de diagnosticar essas limitações** foi uma experiência valiosa sobre o comportamento de sistemas reais sob estresse.

Para o futuro, vemos diversas possibilidades de evolução para esta solução. A implementação de bloqueios transacionais (*locks*) ou serviços de mensageria poderiam resolver o problema de concorrência na alocação de entregadores. Além disso, seria relevante refinar as métricas de escala automática para garantir uma resposta mais agressiva a picos de tráfego repentinos.